

TIENDA PHP

Proyecto Final de Desarrollo de Aplicaciones Web

Documentación técnica completa para la defensa del proyecto

2.º DAW · 2024 / 2025

PHP 8.0+

MySQL

PayPal SDK

Google
OAuth

PHPMailer

MVC propio

Índice de contenidos

1. Descripción general del proyecto	3
2. Arquitectura del proyecto	4
3. Sistema de arranque y enrutamiento	5
4. Base de datos	6
5. Autenticación de usuarios	8
6. Cesta de la compra	10
7. Proceso de pago con PayPal	11
8. Panel de administración	12
9. Sistema de email — PHPMailer + Mailtrap	13
10. Seguridad implementada	13
11. Estructura de ficheros completa	14
12. Guía de instalación rápida	16
13. Decisiones de diseño y posibles mejoras	16

1. Descripción general del proyecto

Tienda PHP es una aplicación web de comercio electrónico desarrollada íntegramente en PHP nativo, sin ningún framework externo, como proyecto final del módulo de **Desarrollo de Aplicaciones Web (2.º DAW)**. El objetivo principal fue demostrar el dominio de la arquitectura MVC, la gestión de sesiones, el acceso a bases de datos con PDO, la integración de servicios de terceros (PayPal y Google OAuth) y las buenas prácticas de seguridad.

El proyecto está inspirado en el repositorio público *pablovacasgarcia/TiendaPHP*, pero reescrito con nombres de clases, métodos y estructura propios, añadiendo funcionalidades adicionales como el login con Google, la integración real del SDK de PayPal y la separación en capa de Servicios y Repositorios.

1.1 Funcionalidades principales

- Catálogo de productos paginado, con filtro por categoría y buscador por nombre.
- Registro de usuarios con verificación por email (PHPMailer + Mailtrap).
- Login clásico con bcrypt y login alternativo mediante **Google OAuth 2.0**.
- Cesta de la compra persistente en sesión, con validación de stock en tiempo real.
- Proceso de compra completo con integración de **PayPal Sandbox**.
- Pedidos almacenados en BD con transacción (cabecera + líneas + descuento de stock).
- Panel de administración: CRUD de productos, categorías y listado de usuarios.
- Sistema de mensajes flash, middleware de acceso y middleware de rol admin.
- Subida de imágenes de producto al servidor.
- Página de instalación automática con usuarios de prueba hasheados.

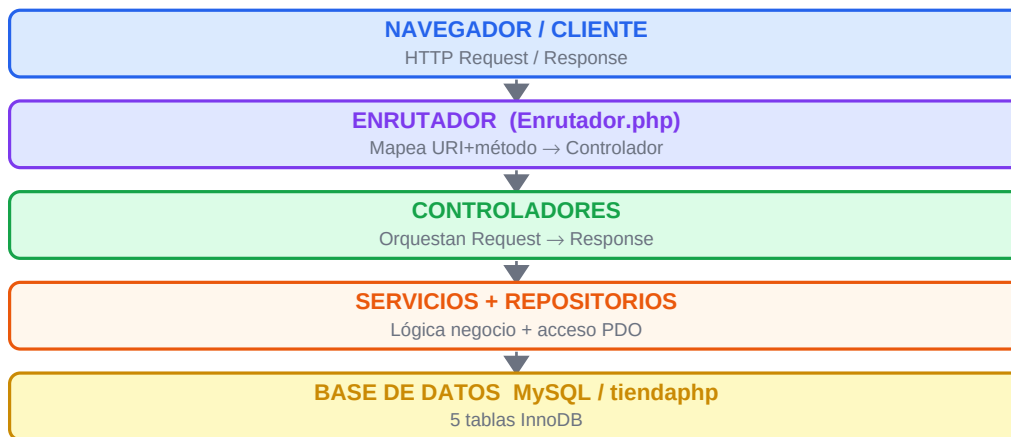
1.2 Tecnologías y herramientas

Tecnología	Versión / Detalle	Uso en el proyecto
PHP	8.0+	Lenguaje principal, lógica servidor
MySQL	5.7+ / MariaDB 10+	Base de datos relacional
PDO	nativo PHP	Acceso a BD con prepared statements
PHPMailer	^7.0	Envío de emails SMTP (Mailtrap sandbox)
PayPal SDK	^2.2	Integración pasarela de pago sandbox
Google OAuth	API v3	Autenticación con cuenta Google
Composer	PSR-4 autoload	Gestión de dependencias y autoloading
XAMPP	Apache + MySQL	Entorno local de desarrollo
CSS propio	sin framework	Estilos de la interfaz

2. Arquitectura del proyecto

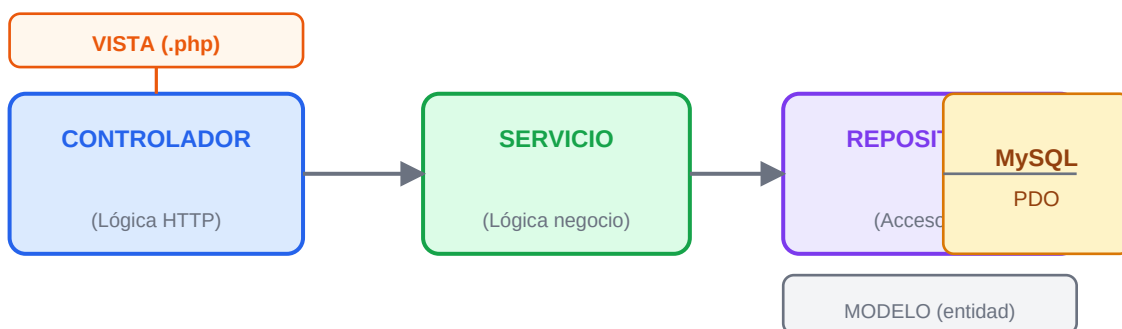
El proyecto sigue el patrón **MVC (Modelo–Vista–Controlador)** adaptado a PHP nativo. Se añaden capas adicionales de **Servicios** (lógica de negocio) y **Repositorios** (acceso a datos), separando claramente las responsabilidades.

2.1 Diagrama de capas



El flujo de una petición HTTP sigue siempre el mismo camino: el navegador envía la petición a **public/index.php** (único punto de entrada), que arranca el bootstrap **src/init.php**. Este carga las variables de entorno, registra el autoloader y ejecuta **Rutas::registrar()**, que delega en el **Enrutador** para encontrar el controlador correcto.

2.2 Diagrama MVC detallado



Cada petición pasa siempre por el mismo camino: **Controlador** → **Servicio** → **Repositorio** → **BD**. Los Modelos son simples estructuras de datos (entidades). Las Vistas son archivos .php que reciben variables del controlador y generan el HTML.

2.3 Responsabilidades de cada capa

Capa	Clase / Carpeta	Responsabilidad
Enrutador	Lib/Enrutador.php	Mapea método+URI a un callable
Controlador	src/Controladores/	Recibe la request, llama al servicio, carga vista
Middleware	src/Middleware/	Comprueba login/rol antes de llegar al controlador
Request	src/Requests/	Valida y sanitiza los datos del formulario
Servicio	src/Servicios/	Lógica de negocio (hashear claves, crear pedidos, etc.)
Repositorio	src/Repositorios/	Consultas PDO a la BD (SELECT, INSERT, UPDATE, DELETE)
Modelo	src/Modelos/	Entidad de datos pura (sin lógica)
Vista	src/Vistas/	Plantilla PHP que genera el HTML
Lib	src/Lib/	Clases de infraestructura: Sesion, Cesta, Enrutador, Mail, OAuth
Utils	src/Utils/	Helpers: Paginador, Paypal, Utilidades (.env)

3. Sistema de arranque y enrutamiento

3.1 Punto de entrada — `public/index.php`

Toda petición HTTP llega a `public/index.php` gracias al fichero `public/.htaccess` que reescribe todas las URLs con `mod_rewrite`. Este fichero simplemente incluye `src/init.php` y ya.

3.2 Bootstrap — `src/init.php`

Es el corazón del arranque. Realiza las siguientes acciones en orden:

- Define constantes de rutas: **RAIZ**, **APP**, **CONFIG**, **PUBLICO**, **URL_BASE**.
- Carga el fichero `.env` mediante `Utilidades::cargar()` que rellena `$_ENV`.
- Ejecuta el autoloader de Composer (`vendor/autoload.php`) para PHPMailer y PayPal SDK.
- Registra un autoloader manual (`spl_autoload_register`) que mapea cada namespace a su carpeta.
- Inicia la sesión PHP si aún no está activa.
- Llama a `Rutas::registrar()` que define todas las rutas y despacha la petición.

3.3 El Enrutador — `Lib/Enrutador.php`

Es un router estático minimalista. Funciona en tres pasos:

- **agregar(método, patrón, callable)**: registra una ruta en un array estático `$rutas[MÉTODO][patrón]`.
- **despachar()**: lee `$_SERVER['REQUEST_METHOD']` y `REQUEST_URI`, elimina la URL base y busca coincidencia exacta. Si no la hay, intenta el patrón `base/:id` extrayendo el último segmento como parámetro dinámico.
- Si ninguna ruta coincide, devuelve HTTP 404 y carga la vista de error.

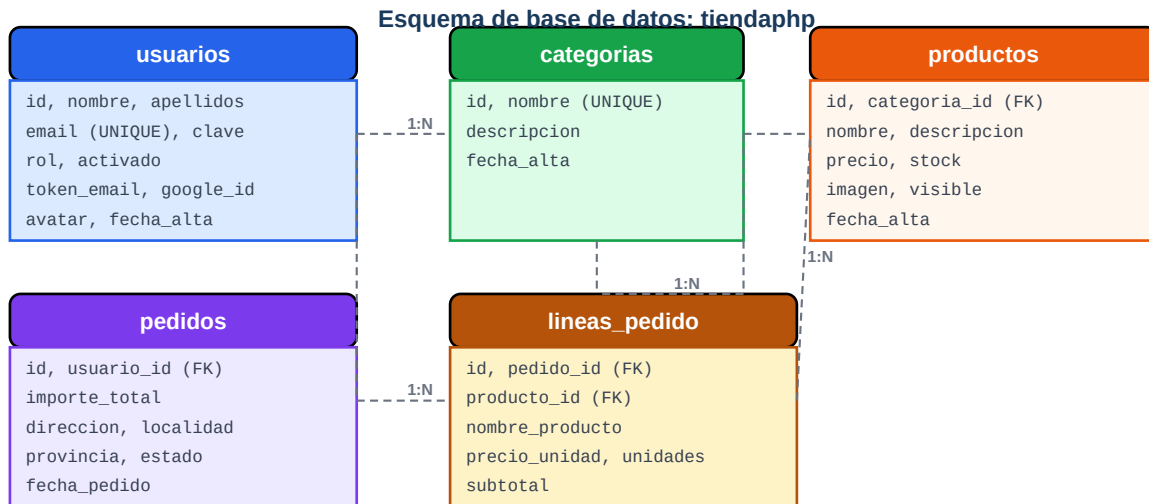
■■ El enrutador solo soporta un parámetro dinámico por ruta (`:id`). Es suficiente para este proyecto y evita la complejidad de un router completo.

3.4 Registro de rutas — `Rutas/Rutas.php`

Centraliza TODAS las rutas de la aplicación. Cada ruta es un closure que instancia el controlador correspondiente y llama al método adecuado. Esto facilita ver de un vistazo toda la navegación del proyecto.

4. Base de datos

4.1 Diagrama de tablas



4.2 Descripción de tablas

Tabla	Campos clave	Relaciones
usuarios	id, email (UNIQUE), clave (bcrypt), rol (ENUM), activado, token_email, google_id, avatar	Referenciada por pedidos (1:N)
categorias	id, nombre (UNIQUE), descripcion	Referenciada por productos (1:N)
productos	id, categoria_id (FK), nombre, precio, stock, imagen, visible	categoria_id → categorias.id (RESTRICT/CASCADE)
pedidos	id, usuario_id (FK), importe_total, direccion, localidad, provincia, estado (ENUM)	usuario_id → usuarios.id (CASCADE)
lineas_pedido	id, pedido_id (FK), producto_id (FK), nombre_producto, precio_unidad, unidades, subtotal	pedido_id → pedidos (CASCADE); producto_id → productos (RESTRICT)

Todos los motores son **InnoDB**, lo que garantiza soporte de claves foráneas y transacciones ACID. El charset es **utf8mb4** para soportar caracteres Unicode completos.

4.3 Conexión PDO — Config/Conexion.php

Implementa el patrón **Singleton**: la primera llamada a `Conexion::abrir()` crea el objeto PDO con los parámetros de conexión; las siguientes devuelven la misma instancia. Esto evita abrir múltiples conexiones en la misma petición.

- Modo de errores: **ERRMODE_EXCEPTION** — lanza excepciones en lugar de errores silenciosos.

- Fetch mode por defecto: **FETCH_ASSOC** — arrays asociativos, no objetos.
- **EMULATE_PREPARES = false** — usa prepared statements reales del servidor MySQL.

4.4 Transacción al crear pedido

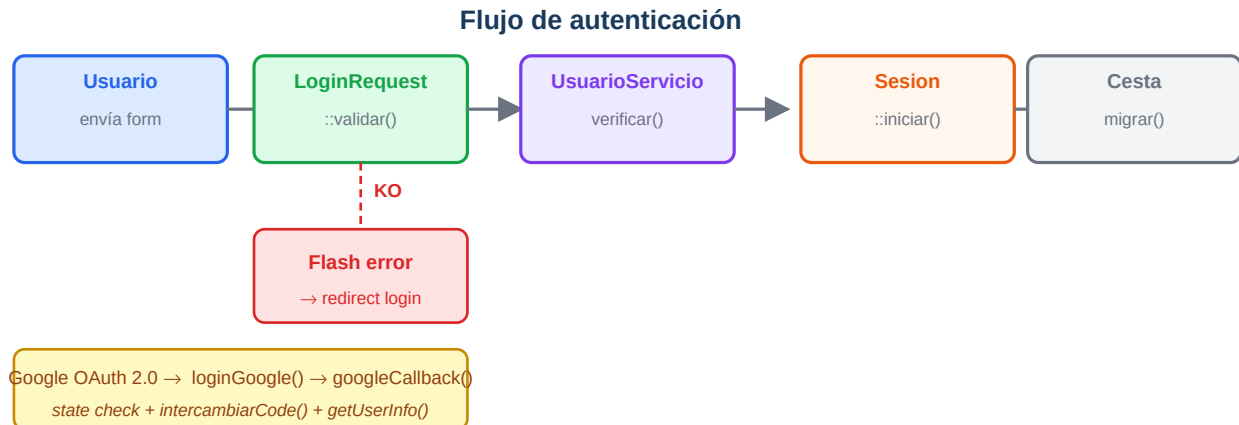
El método **PedidoServicio::crearPedido()** envuelve toda la operación en una transacción PDO para garantizar integridad:

- Inicia transacción con `beginTransaction()`.
- Inserta la cabecera del pedido (*`PedidoRepositorio::insertarCabecera()`*).
- Para cada ítem de la cesta: inserta la línea y descuenta el stock (*`restarStock()`*).
- Si todo va bien: `commit()`. Si falla cualquier paso: `rollback()`.

■■ Gracias a la transacción, si el servidor cae a mitad del proceso no quedará un pedido sin líneas ni stock descontado a medias.

5. Autenticación de usuarios

5.1 Flujo de autenticación



5.2 Registro clásico

El proceso de registro sigue estos pasos:

- El formulario POST llega a **AuthControlador::procesarRegistro()**.
- **RegistroRequest::validar()** comprueba que todos los campos están presentes, que el email tiene formato válido y que las contraseñas coinciden.
- **UsuarioServicio::emailExiste()** consulta la BD para evitar duplicados.
- **UsuarioServicio::registrar()** genera un token aleatorio de 32 caracteres (`bin2hex(random_bytes(16))`), hashlea la contraseña con **bcrypt** y hace el INSERT con `activado = 0`.
- **EnvioMail::confirmacionRegistro()** envía el email con el enlace de activación.
- Al hacer clic en el enlace, **AuthControlador::confirmar()** busca el token, pone `activado = 1` y borra el token de la BD.

5.3 Login clásico y seguridad de contraseñas

UsuarioServicio::verificarCredenciales() busca el usuario por email y llama a `password_verify($clave, $hash)` para comparar. Nunca se almacena ni se compara la contraseña en texto plano.

- Las contraseñas se hashlean con **PASSWORD_BCRYPT** (coste 10 por defecto).
- `bcrypt` incluye un **salt** aleatorio en el propio hash, por lo que dos contraseñas iguales producen hashes diferentes.
- Si la cuenta no está activada, el login es rechazado aunque la contraseña sea correcta.
- Al hacer login, se llama a **Cesta::migrarAlLogin()** para fusionar la cesta de invitado con la del usuario.

5.4 Google OAuth 2.0

Implementado desde cero en **Lib/GoogleOAuth.php** sin librerías adicionales, usando cURL. El flujo OAuth 2.0 con Authorization Code es:

- **loginGoogle()**: genera un *state* aleatorio anti-CSRF, lo guarda en sesión y redirige al usuario a la URL de consentimiento de Google (*accounts.google.com/o/oauth2/v2/auth*).
- Google redirige a **/auth/googleCallback** con el *code* y el *state*.
- Se verifica que el *state* recibido coincide con el de la sesión (protección CSRF).
- **intercambiarCode()** hace un POST a *oauth2.googleapis.com/token* para cambiar el code por un *access_token*.
- **getUserInfo()** llama a la API de Google con el token para obtener email, nombre y foto.
- **UsuarioServicio::procesarLoginGoogle()** hace un *upsert*: si ya existe un usuario con ese *google_id* o email, lo actualiza; si no, lo inserta como nuevo usuario con *activado* = 1 (no necesita verificar email, Google ya lo hizo).

5.5 Gestión de sesión — Lib/Sesion.php

Clase estática con métodos de utilidad para toda la aplicación:

Método	Descripción
iniciar(\$usuario)	Guarda id, nombre, email y rol en \$_SESSION['usuario']
cerrar()	Destruye los datos de sesión del usuario y su cesta
logeado()	Comprueba si existe \$_SESSION['usuario']
esAdmin()	Comprueba que rol === 'admin'
mensaje(\$tipo,\$txt)	Guarda un mensaje flash en sesión
consumirMensaje()	Lee y borra el flash (se muestra una sola vez)
exigirLogin()	Si no hay sesión, redirige a /auth/login
exigirAdmin()	Si no es admin, redirige al inicio con error
redirigir(\$ruta)	header(Location) con la URL base del proyecto

6. Cesta de la compra

La cesta se almacena íntegramente en **sesión PHP** como un array [`id_producto => unidades`]. La clave de sesión varía según si el usuario está autenticado o no:

- **Usuario autenticado:** clave `cesta_u{id}` — personalizada por usuario.
- **Invitado:** clave `cesta_invitado` — compartida en la sesión.

6.1 Métodos de Lib/Cesta.php

Método	Descripción
<code>meterProducto(\$id, \$uds)</code>	Valida stock consultando la BD antes de añadir
<code>cambiarUnidades(\$id, \$uds)</code>	Modifica cantidad; si $uds \leq 0$ llama a <code>quitarProducto()</code>
<code>quitarProducto(\$id)</code>	Elimina el ítem del array de sesión
<code>vaciar()</code>	Resetea el array a []
<code>contenido()</code>	Devuelve array completo con datos del producto y subtotal
<code>importeTotal()</code>	Suma $\text{precio} \times \text{cantidad}$ de todos los ítems
<code>totalUnidades()</code>	Suma de todas las unidades en cesta
<code>migrarAlLogin(\$idUser)</code>	Fusiona <code>cesta_invitado</code> en <code>cesta_u{id}</code> al hacer login

■■ La validación de stock en `meterProducto()` consulta la BD en el momento del añadido, no solo al finalizar. Esto evita que el usuario pueda añadir más unidades de las disponibles aunque el stock cambie mientras navega.

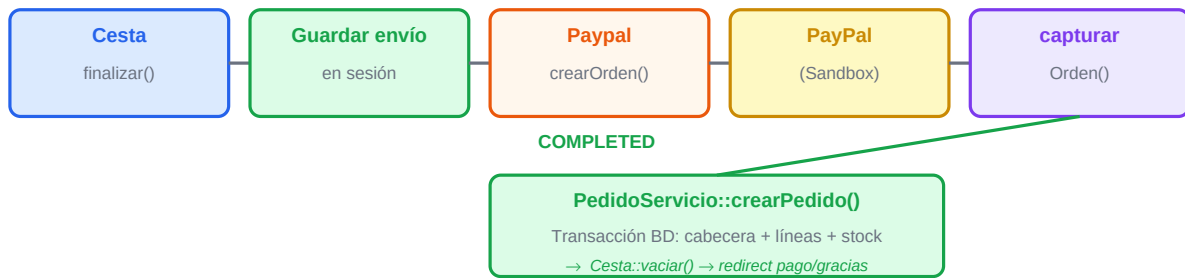
6.2 Migración de cesta al hacer login

Si un usuario añade productos siendo invitado y luego hace login, **migrarAlLogin()** fusiona ambas cestas sumando las unidades. Si un producto ya estaba en la cesta del usuario, se suman las cantidades. La cesta de invitado se elimina de sesión.

7. Proceso de pago con PayPal

7.1 Diagrama del flujo de pago

Flujo de pago PayPal



7.2 Integración PayPal Sandbox

La clase **Utils/ Paypal.php** implementa directamente las llamadas a la API REST de PayPal usando cURL (sin el SDK oficial), lo que permite entender exactamente qué ocurre en cada paso.

- **getAccessToken()**: hace una petición POST a `/v1/oauth2/token` con las credenciales (Client ID + Secret) en formato Basic Auth para obtener un Bearer token temporal.
- **crearOrden(\$total)**: crea una orden de intención *CAPTURE* en `/v2/checkout/orders` con el importe en EUR y las URLs de retorno (*return_url* y *cancel_url*). PayPal devuelve un array con un enlace *approve* al que se redirige al usuario.
- **capturarOrden(\$orderId)**: una vez que el usuario aprueba en PayPal, se llama a `/v2/checkout/orders/{id}/capture` para confirmar el cobro. Si el status es *COMPLETED*, se crea el pedido en BD.

7.3 Flujo completo paso a paso

- Usuario en `/cesta/finalizar`: introduce dirección de envío y pulsa "Pagar".
- **CestaControlador::confirmar()** guarda los datos de envío en sesión y el total de la cesta, llama a `crearOrden()` y redirige al usuario a PayPal.
- El usuario aprueba o cancela en la pantalla de PayPal.
- PayPal redirige a `/pago/exito` con el token de la orden.
- **PagoControlador::exito()** verifica que el token coincide con el guardado en sesión (prevención de falsificación), llama a `capturarOrden()` y si es *COMPLETED* ejecuta `PedidoServicio::crearPedido()`.
- Se vacía la cesta y se redirige a `/pago/gracias`.

8. Panel de administración

El acceso al panel está protegido por **AdminMiddleware**, que internamente llama a `Sesion::exigirAdmin()`. Cualquier usuario sin rol admin es redirigido al inicio con un mensaje de error.

8.1 Funcionalidades del panel

Sección	Operaciones disponibles
Dashboard (/)	Visión general del panel con accesos rápidos
Productos	Listar · Crear (con subida de imagen) · Editar · Eliminar · Activar/desactivar
Categorías	Listar · Crear · Eliminar (con comprobación de productos asociados)
Usuarios	Listar todos los usuarios · Ver rol y estado · Eliminar

8.2 Subida de imágenes

AdminController::guardarProducto() gestiona la subida de imagen con validación de tipo MIME, generación de nombre único con `time()` . `'_'` . `uniqid()` y almacenamiento en `public/uploads/imagenes/`.

8.3 Middleware de acceso

Existen dos middlewares para controlar el acceso a rutas protegidas:

- **AccesoMiddleware**: llama a `Sesion::exigirLogin()`. Se usa en la cesta (finalizar) para requerir que el usuario esté autenticado.
- **AdminMiddleware**: llama a `Sesion::exigirAdmin()`. Se usa en todas las rutas del panel de administración.

9. Sistema de email — PHPMailer + Mailtrap

El envío de emails se realiza con **PHPMailer** usando SMTP. Para el entorno de desarrollo se usa **Mailtrap**, un servicio sandbox que intercepta los emails sin enviarlos realmente, permitiendo ver el resultado en una bandeja de entrada virtual.

Lib/EnvioMail.php lee las credenciales SMTP del fichero `.env` (`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`) y construye un email HTML con el enlace de activación de la cuenta. Basta con cambiar las credenciales en `.env` por las de un SMTP real para enviar correos de verdad.

10. Seguridad implementada

Amenaza	Medida implementada
SQL Injection	Todos los accesos a BD usan PDO con prepared statements y parámetros nombrados
XSS (Cross-Site Scripting)	htmlspecialchars() en todos los datos de usuario antes de INSERT y en vistas
CSRF (OAuth)	Token state aleatorio verificado en el callback de Google OAuth
Contraseñas planas	bcrypt (PASSWORD_BCRYPT) con salt automático; password_verify() para comparar
Acceso no autorizado	Middleware de login y admin en todas las rutas protegidas
Activación de cuentas	Cuentas creadas con activado=0 hasta confirmar email con token único
Falsificación de pago	Se verifica que paypal_order_id en sesión coincide con el token recibido
Rutas de admin	AdminMiddleware antes de cualquier operación del panel

11. Estructura de ficheros completa

La siguiente tabla resume los ficheros más relevantes del proyecto y su función específica:

Fichero / Carpeta	Función
public/index.php	Front controller — único punto de entrada HTTP
public/.htaccess	Reescritura de URLs con mod_rewrite
public/instalar.php	Script de instalación de usuarios de prueba (borrar tras usar)
public/uploads/imagenes/	Imágenes subidas por el administrador
src/init.php	Bootstrap: constantes, .env, autoloader, sesión, rutas
src/Rutas/Rutas.php	Registro centralizado de todas las rutas
src/Lib/Enrutador.php	Router HTTP minimalista con soporte de parámetro dinámico
src/Lib/Sesion.php	Gestión de sesión, flash messages y redirección
src/Lib/Cesta.php	Carrito en sesión con validación de stock
src/Lib/EnvioMail.php	Envío SMTP con PHPMailer
src/Lib/GoogleOAuth.php	Cliente OAuth 2.0 para Google (cURL propio)
src/Controladores/	Uno por sección: Auth, Home, Producto, Cesta, Pago, Admin
src/Servicios/	Lógica de negocio: Usuario, Producto, Categoría, Pedido
src/Repositorios/	Acceso PDO a BD: Usuario, Producto, Categoría, Pedido
src/Modelos/	Entidades de datos: Usuario, Producto, Categoría, Pedido
src/Requests/	Validación de formularios: Login, Registro, Producto, Categoría
src/Middleware/	AccesoMiddleware (login), AdminMiddleware (admin)
src/Utils/Paypal.php	Cliente REST de PayPal Sandbox con cURL
src/Utils/Paginador.php	Cálculo de páginas para el catálogo
src/Utils/Utilidades.php	Carga del fichero .env en \$_ENV
src/Vistas/	Plantillas PHP organizadas por sección
config/Conexion.php	Singleton PDO para la conexión a MySQL
config/google.php	Credenciales de la app de Google Cloud Console
miSQL/tiendaphp.sql	Script SQL completo: tablas + datos de ejemplo
.env	Variables de entorno: BD, SMTP, PayPal, URL base
composer.json	Dependencias: phpmailer/phpmailer, paypal/paypal-server-sdk

12. Guía de instalación rápida

- Copiar el proyecto en C:\xampp\htdocs\ProyectoTiendaPHP.
- Ejecutar composer install en la raíz del proyecto.
- Crear la base de datos importando miSQL/tiendaphp.sql en phpMyAdmin.
- Rellenar el fichero .env con las credenciales SMTP y PayPal.
- Visitar <http://localhost/ProyectoTiendaPHP/public/instalar.php> una sola vez.
- Acceder a <http://localhost/ProyectoTiendaPHP/>.

Usuario	Contraseña	Rol
admin@tiendaphp.com	admin123	admin
cliente@tiendaphp.com	cliente123	cliente

13. Decisiones de diseño y posibles mejoras

13.1 ¿Por qué sin framework?

El objetivo didáctico del proyecto era demostrar que se comprende el funcionamiento interno de un framework PHP, no simplemente usarlo. Construir el enrutador, el sistema de sesiones y la arquitectura MVC desde cero permite entender qué hace Laravel o Symfony "por debajo".

13.2 ¿Por qué separar Servicios y Repositorios?

En un MVC clásico, los Modelos mezclan lógica de negocio y acceso a datos. Separar **Servicios** (qué hace el sistema) de **Repositorios** (cómo accede a la BD) facilita el testing unitario y el cambio de motor de BD sin tocar la lógica.

13.3 Posibles mejoras

- Añadir tokens CSRF en los formularios POST para proteger contra ataques CSRF.
- Implementar rate limiting en el login para prevenir fuerza bruta.
- Mover las credenciales de config/Conexion.php a leerlas del .env.
- Añadir historial de pedidos para el cliente en su perfil.
- Implementar paginación en el panel de administración.
- Tests unitarios para los Servicios con PHPUnit.
- Añadir recuperación de contraseña por email.